# Übungsblatt 9: Wrap-Up & Finale (State Machine)

**Mikrocomputertechnik 1**

**Musterlösung**

## Zielsetzung

In diesem letzten Übungsblatt schließt sich der Kreis. Sie wiederholen die Werkzeugkette (Toolchain) und die Übersetzungsmuster von C zu Assembler. Anschließend nutzen Sie Ihre Kenntnisse in I/O, Kontrollfluss und Logik, um im Ripes-Simulator ein interaktives Mini-Projekt (Code-Schloss) als Endliche Zustandsmaschine (FSM) zu programmieren.

## Aufgabe 1: Die Toolchain

Wenn Sie in der Industrie ein C-Programm mit `gcc main.c` kompilieren, ruft das System im Hintergrund vier verkettete Werkzeuge auf.

### Die vier Werkzeuge

Benennen Sie diese vier Werkzeuge in der korrekten Reihenfolge.

### Aufgabe des Linkers

Welche spezifische Aufgabe übernimmt das letzte Werkzeug in dieser Kette? Warum ist das Programm nach Schritt 3 (Object-File `.o`) noch nicht ausführbar?

### Ihre Lösung

*(Notieren Sie Ihre Antworten hier.)*

**Musterlösung Aufgabe 1**

Die vier Werkzeuge:

1. Preprocessor
2. Compiler

3. Assembler
4. Linker

Aufgabe des Linkers:

- Der Linker klebt alle Object-Files (`.o`) und externe Bibliotheken (z. B. `libc`) zu einem ausführbaren Programm (z. B. `.elf` / `.exe`) zusammen.
- Nach dem Assembler fehlen oft noch finale Adressen (z. B. für `jal` zu externen Funktionen oder globale Daten). Der Linker trägt diese Adressen nach.

## Aufgabe 2: C-to-Assembly (Klausurvorbereitung)

In der Klausur wird erwartet, dass Sie typische C-Kontrollstrukturen in RISC-V-Assembler übersetzen können. Übersetzen Sie die folgende `while`-Schleife.

Hinweis: Variable `i` liegt in `s0`. Nutzen Sie das invertierte Bedingungsmuster (`bge`), wie es ein Compiler typischerweise tut.

```c
int i = 0;
while (i < 10) {
    i++;
}
```

### Ihre Lösung

*(Fügen Sie Ihren Assembler-Code hier ein.)*

**Musterlösung Aufgabe 2**

```
    # Initialisierung
    li s0, 0            # i = 0
    li t0, 10           # Grenze = 10

While_Start:
    # Invertierte Abbruchbedingung (if i >= 10, break)
    bge s0, t0, While_End

    # Body
    addi s0, s0, 1      # i++

    j While_Start

While_End:
    # Weiter nach der Schleife
```

## Aufgabe 3: Pipelining und Hazards (Klausurvorbereitung)

Betrachten Sie das folgende Code-Snippet in der 5-stufigen RISC-V-Pipeline:

```
lw t0, 0(a0)
add t1, t0, t0
```

### Hazard-Typ

Welches Hazard-Problem entsteht bei dieser Befehlsfolge?

### Forwarding vs. Stall

Kann dieses Problem allein durch Forwarding (Bypassing) ohne Zeitverlust gelöst werden? Erklären Sie, wie die Hardware im Pipeline-Raster (IF, ID, EX, MEM, WB) darauf reagieren muss.

### Ihre Lösung

*(Notieren Sie Ihre Antworten hier.)*

**Musterlösung Aufgabe 3**

Hazard-Typ:

- Es entsteht ein Load-Use Hazard: `add` benötigt `t0` sofort, aber `lw` muss den Wert erst aus dem Arbeitsspeicher holen.

Forwarding vs. Stall:

- Nein, reines Forwarding reicht nicht.
- Die RAM-Daten stehen erst am Ende der MEM-Stufe von `lw` bereit; `add` bräuchte sie schon am Anfang seiner EX-Stufe.
- Die Hardware erzwingt einen Stall: `add` bleibt in ID stehen, eine Bubble (NOP) wird in EX eingeschoben — ein Takt Penalty. Danach kann Forwarding von MEM nach EX greifen.

## Aufgabe 4: Finale — State Machine (Code-Schloss)

Bauen Sie in Ripes eine Endliche Zustandsmaschine (FSM) als Code-Schloss. Fügen Sie im I/O-Tab ein **D-Pad** (typisch `0xF0000000`) und eine **LED Matrix** (typisch `0xF0000020`) hinzu. Basisadressen unten links ablesen.

### FSM-Logik

- Zustandsvariable: Register `s0` für den aktuellen Zustand (0, 1, 2 oder 3).
- Geheimcode: Tasten in der Reihenfolge **OBEN** → **UNTEN** → **RECHTS**.

Verhalten:

| Zustand | LED-Farbe | Hex | Erwartete Taste | Sonst |
|---|---|---|---|---|
| 0 | Rot | `0x00FF0000` | OBEN → 1 | bleiben |
| 1 | Gelb | `0x00FFFF00` | UNTEN → 2 | zurück auf 0 |
| 2 | Blau | `0x000000FF` | RECHTS → 3 | zurück auf 0 |
| 3 | Grün | `0x0000FF00` | (offen) | bleibt für immer |

**D-Pad in Ripes (kein Bitmasken-Modell):** Jede Richtung ist ein **eigenes 32-Bit-Register** (LSB = gedrückt):

| Richtung | Offset vom D-Pad-Basis |
|---|---|
| OBEN (UP) | `+0` |
| UNTEN (DOWN) | `+4` |
| LINKS (LEFT) | `+8` |
| RECHTS (RIGHT) | `+12` |

**Entprellung / Release:** Nach erkanntem Tastendruck **warten, bis alle Richtungen wieder losgelassen sind** (Register = 0). Eine reine Delay-Schleife reicht nicht — sonst wertet der Folgezustand denselben Klick als falsche Taste.

### Ihre Lösung

*(Fügen Sie Ihren Assembler-Code für Ripes hier ein.)*

**Musterlösung Aufgabe 4**

```
.text
    li s1, 0xF0000000   # D-Pad-Basis
    li s2, 0xF0000020   # LED Matrix
    li s0, 0            # State = 0

Loop:
    beq s0, zero, State_0
    li t0, 1
    beq s0, t0, State_1
    li t0, 2
    beq s0, t0, State_2
    li t0, 3
    beq s0, t0, State_3
```

```
    j State_0

# --- State 0: wartet auf OBEN (+0) ---
State_0:
    li t1, 0x00FF0000
    sw t1, 0(s2)
    lw t2, 0(s1)        # UP
    beq t2, zero, Loop
    jal ra, WaitRelease
    li s0, 1
    j Loop

# --- State 1: wartet auf UNTEN (+4) ---
State_1:
    li t1, 0x00FFFF00
    sw t1, 0(s2)
    lw t2, 4(s1)        # DOWN erwartet
    bne t2, zero, S1_Ok
    lw t3, 0(s1)
    bne t3, zero, State_Fail
    lw t3, 8(s1)
    bne t3, zero, State_Fail
    lw t3, 12(s1)
    bne t3, zero, State_Fail
    j Loop
S1_Ok:
    jal ra, WaitRelease
    li s0, 2
    j Loop

# --- State 2: wartet auf RECHTS (+12) ---
State_2:
    li t1, 0x000000FF
    sw t1, 0(s2)
    lw t2, 12(s1)       # RIGHT erwartet
    bne t2, zero, S2_Ok
    lw t3, 0(s1)
    bne t3, zero, State_Fail
    lw t3, 4(s1)
    bne t3, zero, State_Fail
    lw t3, 8(s1)
    bne t3, zero, State_Fail
    j Loop
S2_Ok:
    jal ra, WaitRelease
    li s0, 3
    j Loop
```

```
# --- State 3: offen ---
State_3:
    li t1, 0x0000FF00
    sw t1, 0(s2)
    j Loop

State_Fail:
    jal ra, WaitRelease
    li s0, 0
    j Loop

# Warten, bis alle D-Pad-Richtungen losgelassen sind
WaitRelease:
Wr_Loop:
    lw t4, 0(s1)
    bne t4, zero, Wr_Loop
    lw t4, 4(s1)
    bne t4, zero, Wr_Loop
    lw t4, 8(s1)
    bne t4, zero, Wr_Loop
    lw t4, 12(s1)
    bne t4, zero, Wr_Loop
    ret
```